Projet Informatique

L2 Informatique

2023/2024

- *Fait par :*
- *ALFARES Mohamed*
- *OUANENNAS Rahma*
- *IBERSIENE Amira*



Concept du jeu :

- Deux joueurs à la fois.
- Un de trois modes.



Règles du jeu :

Mode	Easy	Médium	Hard
Collision contre le mur	Rien ne se passe	Meurt	Meurt
Collision tête / tête	Rien ne se passe	Meurt	Meurt
Collision tête / corps	Rien ne se passe	Meurt	Perd sa longueur
Collision lui-même	Rien ne se passe	Rien ne se passe	Longueur égale Zéro

- Dans le Easy mode si un des deux joueurs mange 50 pommes , le jeu se finit.
- Dans le Hard mode, collision tête / corps l'adversaire gagne la langueur du joueur perdant.
- Un bonus toutes les cinq pommes, soit le joueur perd la moitié de sa longueur, soit il la gagne.

Les problèmes que nous avons rencontrés :

- 1- la longueur du serpent nous a fait problème, car en utilisant une liste statique, on ne sait pas déjà le max de la longueur du serpent, alors la meilleure solution était l'utilisation de la liste dynamique (vu au deuxième semestre avec monsieur Clavier).
- 2- nous avons trouvé un problème au niveau de l'initialisation du temps à 0 quand on clique start, nous avons le temps qui commence le moment d'exécution du programme et ne pas le moment du début du jeu , car nous initialisions le start time dans la fonction draw(), mais finalement nous devions l' initialiser dans le background start juste après "start", et l'affichage dans le board .

Les problèmes que nous avons rencontrés :

```
if (mouseX >= 300 && mouseX <= 500 && mouseY >= 450 && mouseY <= 500) {
  SColor = color(123, 127);
  if (mousePressed && mouseButton == LEFT) {
    if(order[0] == null || order[1] == null){ // if the troisieme element

    order[2] = "";}
    else{order[2] = "S";
    startTime = millis();
  son.play();}
```



initialisation start time

```
drawFont(20, 770, 40, str((millis() - startTime) / 1000), color(0));
drawFont(15, 700, 40, "PLAYED TIME : ", color(0));
```



Affichage start time

Les problèmes que nous avons rencontrés :

- 3- la division de plateau du jeu en cellule, au début, le mouvement du serpent était aléatoire dans le plateau car le serpent ne pouvait pas manger la pomme, et après plusieurs manipulations, nous avons trouvé que l'erreur était au niveau de la division des cellules de plateau du jeu.(vu avec monsieur Maxime Maria).

```
sizeCel = 20;
widthUnities = width / sizeCel;
heightUnities = height/sizeCel;
for (int i = 0; i < width; i++) {
  for (int j = 3; j < height; j++) {
    if ( (i +j) % 2 == 0) {
      fill(165, 205, 0);
    } else {
      fill(187, 225, 0);
    }
    noStroke();
    rect(i*sizeCel, j*sizeCel, sizeCel, sizeCel);
  }
  stroke(187, 225, 0);
  line(0, 60, width, 60);
}
```

Les problèmes que nous avons rencontrés :

- 4- l'implémentation des conditions de collision était super compliqué à gérer, et elle nous a pris du temps pour organiser toutes les conditions, tel que la collision du Cpu contre le mur, donc nous avons créé la fonction testColliding() qui initialise la variable testing avec la valeur de la collision, puis la fonction checkIn() qui détermine le gagnant et le perdant.
- 5- Le CPU, quand nous avons décidé de faire le jeu Serpent, nous n'avions pas réfléchi à faire un CPU, et quand monsieur Vaccon nous a demandé un CPU, nous nous sommes mis une semaine pour trouver le meilleur mécanisme pour faire marcher le CPU.

Le mécanisme du CPU :

- Quand le X du CPU est supérieur à celui de la pomme, donc le CPU se dirige à gauche et l'inverse à droite.

- Quand le Y du CPU est supérieur à celui de la pomme, donc le CPU se dirige en haut et l'inverse en bas.

```
void move(PVector positionApple) {
  if (cpuPos.x < positionApple.x) {
    cpuDirection = new PVector(1, 0);
  }
  if (cpuPos.y < positionApple.y) {
    cpuDirection = new PVector(0, 1);
  }
  if (cpuPos.x > positionApple.x) {
    cpuDirection = new PVector(-1, 0);
  }
  if (cpuPos.y > positionApple.y) {
    cpuDirection = new PVector(0, -1);
  }
}
```

Le mécanisme du Serpent :

```
void update(ArrayList <PVector> snake, int snakeSpeed, PVector snakeDirection, int snakeLength, PVector snakePos) {
  if (frameCount % snakeSpeed == 0) {
    snake.add(new PVector(snakePos.x, snakePos.y));
    while (snake.size() > snakeLength) {
      snake.remove(0);
    }
    snakePos.add(snakeDirection);
  }
}
```

D'abord nous avons fait une boucle FOR pour dessiner le Serpent, mais il se déplaçait horizontalement, donc nous avons trouvé ce mécanisme qui marche en fonction de la vitesse du Serpent, il ajoute la position actuelle du Serpent, et quand il dessine le Serpent il supprime la dernière pièce du Serpent pour qu'il n'ajoute pas en surplus, et il met à jour la direction du Serpent.

Fin du jeu :



Plan du travail :

- La majorité du travail a été fait en groupe, pendant 15 jours, nous avons travaillé Full-Time, puis nous avons distribué des taches concernant la décoration du jeu, commenter le code, faire le rapport, être en groupe et partager les idées, impliquer ces idées.

Merci pour votre attention.... ❤️

